

Task → Agent Execution → Step-wise Uncertainty Estimation → Predicted Failure Point
→ {Targeted Repair / Full Restart / Random Repair / Oracle Repair} → Evaluation & Causal Analysis

Proposed Research Pipeline

1. Task and Benchmark Selection

HotpotQA

- Multi-hop question answering.
- The agent often needs to retrieve information from multiple sources.
- Eg. - What nationality is the author of the novel that inspired the film Blade Runner?

2. Agent Framework Selection and Trajectory Generation

LLM -

- Qwen2.5-7B-Instruct

Because they are open-weight models and give access to token probabilities and log probabilities, which are useful for uncertainty estimation.

Instruct model because the agent needs to follow instructions and perform tasks step by step.

Agent Framework - ReAct. Can use LangGraph to implement.

Store complete ReAct traces, including thoughts, tool calls, observations, intermediate outputs, and log probabilities. Can store as a JSON file.

3. Step-Level Uncertainty Estimation

Token Entropy - Looks at the probability distribution of the model's next tokens/actions.

Perplexity - Measures how surprised the model is by its own generated output.

Maximum Token Probability - Uses the probability of the action chosen by the model.

Self-Consistency

- Run the same step multiple times.
- If the model gives similar outputs every time, uncertainty is low.
- If outputs differ significantly, uncertainty is high.

Verbalised Confidence/LLM as a judge - Explicitly ask the model how confident it is in its decision and ask it to rate.

Mathematical:

- Token Entropy
- Perplexity
- Maximum Token Probability

Non-mathematical:

- Self-Consistency
- Verbalised Confidence

4. Ground-Truth Error Annotation

- Identify the true erroneous step in failed trajectories to obtain oracle error labels.

5. Error Localisation

- Choose the step with the highest uncertainty as the predicted broken step.
- Top-k Rule - Instead of selecting only one step, choose the top-k uncertain steps.
- Earliest-Above-Threshold Rule - Sometimes the actual error occurs earlier, but uncertainty becomes large only later.

6. Repair Module and Re-run

- Implement and compare four repair strategies:
 - **Full Restart:** Re-execute the entire trajectory.
 - **Random-Step Repair:** Re-execute a randomly selected step.
 - **Uncertainty-Targeted Repair:** Re-execute the step with the highest uncertainty.
 - **Oracle-Targeted Repair:** Re-execute the true erroneous step using ground-truth labels.

Example -

```
{
  "step1": "125 million",
  "step2": "40 trillion",
  "step3": "320000",
  "step4": "$320000"
}
```

Repair Step-2 -

```
{
  "step1": "125 million"
}
```

Give Agent information as such: You have already completed Step 1.

Current state: Population = 125 million. Please continue from Step 2 onward.

The agent then generates: Step 2 → Step 3 → Step 4 again.

Small Nudge - Rerunning a step exactly as before may reproduce the same error. We can give the model a small, identical retry signal (or increase the temperature) for every repair strategy so that it explores alternative solutions fairly.

7. Evaluation

- Evaluate all repair strategies using:
 - Final task accuracy - Did the repaired trajectory solve the task correctly?
 - Error localisation accuracy - Did the uncertainty method identify the true broken step?
 - Repair success rate - Among failed trajectories, how many were successfully fixed?
 - Token cost - How many tokens were spent to recover from the failure?
 - Latency/runtime - How long did recovery take?
 - Number of tool calls - How many external tool/API calls were needed? - Dibo kina janina